

Deep Learning Image Classification Report

Jacob Jones

Introduction

This report discusses the design, training, and evaluation of Multilayer Perceptrons (MLPs) and Convolutional Neural Networks (CNNs) on two benchmark image classification datasets: MNIST (handwritten digits) and CIFAR-10 (natural images across 10 classes). All models were implemented in PyTorch and trained using Google Colab T4 GPU. The main objectives of this project were to evaluate how architectural depth affects performance on simple vs. complex image tasks, compare the inductive bias of MLPs versus CNNs on spatially structured data, and identify the hyperparameter combinations that consistently yield the best validation accuracy.

Datasets and Data Handling

Both datasets were loaded using *torchvision.datasets* and normalized using dataset-standard statistics. MNIST images (28x28 grayscale) were normalized with mean = 0.1307 and std = 0.3081. CIFAR-10 images (32x32 RGB) were normalized per-channel with means of (0.4914, 0.4822, 0.4465) and standard deviations of (0.2470, 0.2435, 0.2616). No data augmentation was applied whatsoever.

Dataset Splits used for All Experiments

Dataset	Train Split	Validation Split	Test Split	Classes
MNIST	50,000	10,000	10,000	10
CIFAR-10	45,000	5,000	10,000	10

A *precachedDataset* wrapper was used on GPU runs meaning the entire dataset was loaded directly onto the GPU's memory at the start instead of the CPU-GPU transfer on every batch. This reduced per-epoch time drastically, specifically on smaller datasets like MNIST.

Multilayer Perceptrons (MLPs)

All MLP variants flatten the output image into a 1-D vector (784 for MNIST, 3072 for CIFAR-10), pass it through stacked Linear+ReLU+Dropout blocks, and produce raw logits for 10 classes. CrossEntropyLoss applies softmax internally, so no explicit activation is used on the output layer. Three depths were evaluated:

MLP Architecture Configurations

Variant	Hidden Layer Sizes	Parameters (MNIST)	Parameters (CIFAR-10)
Shallow	[128]	102K	395K
Medium	[512, 256, 128]	535K	1.83M

Deep	[1024, 512, 256, 128, 64]	1.18M	3.54M
-------------	---------------------------	-------	-------

Convolutional Neural Networks

Two CNN variants were evaluated. Both used 3x3 convolutional kernels with stride = 1 and padding = 1, Batch Normalization after each convolution, and MaxPool2d(2) for downsampling. The classifier head is built dynamically from the flattened feature map, making the same architecture valid for both MNIST and CIFAR-10 inputs.

CNN Architecture Configurations

Variant	Conv. Layers	Filters	FC Head	BN
Simple	2 conv + 2 pool	8, 8	Flat → 10	Yes
Enhanced	3 conv + 2 pool	16 → 32 → 64	Flat → 10	Yes

The Simple CNN intentionally uses very few filters to serve as a lower-bound baseline. The Enhanced CNN progressively doubles the number of filters at each later, allowing it to learn increasingly abstract representations.

Hyperparameter Search

Ten meaningful configurations were evaluated per model type (MLP & CNN), chosen to cover the most informative regions of the hyperparameter space while avoiding exhaustive grid search. For MLPs, the search space included learning rate = {0.01, 0.001, 0.0001}, batch size = {64, 128, 256}, optimizer = {Adam, SGD}, and dropout = {0.2, 0.5}. For CNNs, the Adam optimizer was fixed per the project instructions, and weight decay = {0, 1e-4, 5e-4} was tuned instead of dropout. Training used a maximum of 10 epochs with early stopping and patience = 3. This was done mainly in the best interest of runtime. Higher epochs (~20 epochs) and more patience (5) caused unnecessary increased runtimes. The 10 epochs with 3 patience allowed for more efficient and very similar, if not near identical accuracy. The best configuration per model was selected by validation accuracy, then retrained on the combined train+validation set for final testing.

MLP Hyperparameter Configurations

Configuration	LR	Batch	Optimizer	Dropout
1	0.001	64	Adam	0.2
2	0.001	128	Adam	0.2
3	0.001	256	Adam	0.2

4	0.001	64	Adam	0.5
5	0.01	64	Adam	0.2
6	0.01	256	Adam	0.5
7	0.0001	64	Adam	0.2
8	0.001	64	SGD	0.2
9	0.01	64	SGD	0.2
10	0.01	256	SGD	0.5

CNN Hyperparameter Configurations

Configuration	LR	Batch	Optimizer	Dropout
1	0.001	64	Adam	0
2	0.001	128	Adam	1e-4
3	0.001	256	Adam	1e-4
4	0.001	64	Adam	5e-3
5	0.01	64	Adam	0
6	0.01	256	Adam	1e-4
7	0.01	64	Adam	5e-3
8	0.0001	64	Adam	0
9	0.0001	256	Adam	1e-4
10	0.01	32	Adam	1e-4

Results

The following four tables report test-set accuracy and runtime for the best-performing configuration per architecture. The mean and standard deviation columns reflect the distribution of validation accuracies across all 10 hyperparameter configurations, giving a sense of model sensitivity to the hyperparameter choice.

Table 1: MNIST Results (MLPs)

Architecture	LR	Batch	Opt	Dropout	Val Acc (mean±std)	Test Acc	Runtime
MLP (shallow)	0.01	64	SGD	0.2	0.9648±0.0105	0.9794	26.5s
MLP (medium)	0.01	64	SGD	0.2	0.9654±0.0198	0.9800	31.7s
MLP (deep)	0.001	256	Adam	0.2	0.9452±0.0607	0.9817	16.0s

Table 2: CIFAR-10 Results (MLPs)

Architecture	LR	Batch	Opt	Dropout	Val Acc (mean±std)	Test Acc	Runtime
MLP (shallow)	0.001	256	Adam	0.2	0.4691±0.0795	0.5157	9.1s
MLP (medium)	0.0001	64	Adam	0.2	0.4505±0.1400	0.5516	34.2s
MLP (deep)	0.0001	64	Adam	0.2	0.4166±0.1606	0.5506	44.0s

Table 3: MNIST Results (CNNs)

Architecture	LR	Batch	Weight Decay	Val Acc (mean±std)	Test Acc	Runtime
CNN (simple)	0.01	256	0.0001	0.7469±0.0851	0.8256	18.7s
CNN (enhanced)	0.001	128	0.0001	0.9830±0.0077	0.9910	24.4s

Table 4: CIFAR-10 Results (CNNs)

Architecture	LR	Batch	Weight Decay	Val Acc (mean±std)	Test Acc	Runtime
CNN (simple)	0.01	256	0.0001	0.3143±0.0337	0.3557	12.6s
CNN (enhanced)	0.001	64	0	0.6437±0.0599	0.6998	38.5s

MLP Performance on MNIST vs. CIFAR-10

MLPs performed well on MNIST, with all three depths achieving test accuracies ranging from 97.94% to 98.17%. The medium architecture seemed to be the best performer, though marginally compared to deep, which suggests that for a relatively simple dataset like MNIST, moderate depth provides sufficient capacity without overfitting. The deep MLP scored slightly lower than medium (~0.5%), likely because the additional layers introduced more parameters that were harder to optimize with only 10 epochs available. The low standard deviation across

the configurations indicates that MNIST MLPs are not particularly sensitive to hyperparameter choice; almost any reasonable combination converges to near-peak accuracy.

On CIFAR-10, all MLPs plateaued at the 50-55% range, significantly below the performance achieved by CNNs. The fundamental limitation is that flattening a 32x32x3 image destroys all spatial structure: adjacent pixels that share color gradients, edges, and textures are shuffled into an unordered vector. The MLP must learn every spatial relationship from scratch through dense connections, which is both sample-inefficient and prone to overfitting on a dataset as complex as CIFAR-10. The higher standard deviation across configurations reflects this sensitivity; hyperparameters matter more when the task is harder, and the model is not well-suited to the data structure.

CNN Performance and the Value of Depth

The most striking result is the gap between the Simple and Enhanced CNNs, particularly on CIFAR-10: 35.57% vs. 69.98% test accuracy. This 34-point gap can be attributed to two factors. First, the Simple CNN uses only 8 filters per layer, which is insufficient to represent the diversity of visual patterns in natural images. With only 8 filters, the network can learn at most 8 distinct feature detectors at each spatial scale, sharply limiting its representational capacity. Second, the Enhanced CNN's progressively increasing filter count (16→32→64) mirrors standard design principles: early layers detect low-level features (edges, colors) with fewer filters, while deeper layers combine these into more complex patterns requiring more filter diversity.

On MNIST, the gap is narrower (82.56% vs. 99.10%) but still substantial. Even MNIST, a grayscale dataset, benefits from richer feature extraction. The Simple CNN's poor MNIST performance (82.56%) is surprisingly low, given the task's simplicity; it underperforms all three MLP variants. This is explained by the 8-filter bottleneck: the 2x2 MaxPool after each layer rapidly reduces spatial resolution (28→14→7), and with only 8 channels, the final feature map has very limited information. By contrast, the Enhanced CNN's third convolutional layer (without pooling) retains spatial resolution while boosting channel depth to 64, giving the classifier head a much richer input.

Hyperparameter Analysis

Learning rate. Across both MLPs and CNNs, $\text{lr}=0.001$ with Adam was the most consistently reliable choice on MNIST. On CIFAR-10, $\text{lr}=0.0001$ performed best for MLPs, likely because the harder task benefits from more conservative gradient updates that prevent early overshooting. For CNNs, $\text{lr}=0.01$ with Adam worked well, as Batch Normalization stabilizes the gradient scale and permits larger learning rates.

Optimizer. Adam outperformed SGD in the large majority of configurations. The one notable exception was the shallow MLP on CIFAR-10, where SGD achieved the best test accuracy (51.57%). This can be explained by SGD's implicit regularization effect: its noisier gradient updates act as a form of regularization that prevents the shallow model, which has limited capacity, from memorizing spurious correlations. Adam's adaptive learning rates can sometimes overfit in low-capacity settings.

Batch size. Larger batch sizes (256) were beneficial for CNNs on GPU, primarily because they improve GPU utilization and produce more stable gradient estimates for BatchNorm layers (which compute statistics per batch). For MLPs, batch size had a smaller effect on accuracy, with the differences generally within the standard deviation range across configurations.

Dropout. Dropout=0.2 consistently outperformed dropout=0.5 for MLPs. Heavy dropout (0.5) was too aggressive for models trained for only 10 epochs; the models did not have enough training time to compensate for dropping half their activations per forward pass.

Weight decay. A small weight decay of $1e-4$ helped CNN perform slightly on MNIST but had mixed results on CIFAR-10. The Enhanced CNN actually performed best with no weight decay on CIFAR-10, suggesting that BatchNorm already provides sufficient implicit regularization for this architecture, and additional L2 penalty may limit model capacity on a harder task.

Key Challenge: Runtime

The main challenge I seemed to face throughout this project was condensing my runtime to a manageable duration while still maintaining the functionality and accuracy of the program. The first steps I took to lower runtime was using *torch.compile* to cut per-epoch time by 20-40%. This made a difference, but it was not enough at the time because my program took several hours (4-5 hours) for one execution.

The three main changes I made that made noticeable changes to the runtime duration was lowering my epochs from 20 $\rightarrow$ 10, lowering the patience from 5 $\rightarrow$ 3, and most importantly, caching the datasets to the GPU as soon as the program starts, instead of at the start of each combination change. This saved several hours of runtime alone. This works by making the *num_workers*, multiprocessing, and prefetching irrelevant to the program because there is no constant CPU-to-GPU transfer of information while the program is running. One large transfer is made at the start of MNIST and CIFAR-10 datasets execution, and after that point all of the data is already in the GPU's ram which reduces the runtime down to 45-60mins on average. These changes made negligible effects to the accuracy of the program, epoch/patience made more of a difference in accuracy than precaching the datasets did, neither caused significant negative change however.

Conclusion

This project demonstrated the limitations of MLPs on spatially structured image data and the strong advantage CNNs hold through their translation-equivariant, parameter-efficient design. On MNIST, all architectures performed well (97%+), but on CIFAR-10, CNNs outperformed MLPs by 15–20 percentage points on the enhanced model. The Enhanced CNN was the best overall model, achieving 99.10% test accuracy on MNIST and 69.98% on CIFAR-10, the latter nearly doubling the Simple CNN's accuracy through deeper, wider feature extraction. Among hyperparameters, Adam with $\text{lr}=0.001\text{--}0.01$ and moderate dropout (0.2) was the most reliable combination for MLPs. For CNNs, $\text{lr}=0.01$ with small or no weight decay and larger batch sizes (128–256) performed best. Future work could extend these experiments with data augmentation, residual connections, and learning rate scheduling to further improve CIFAR-10 accuracy.